

بسم الله الرحمن الرحيم

آزمایشگاه معماری

احسان محترم: 401106458 -- روزبه معینی: 401106544

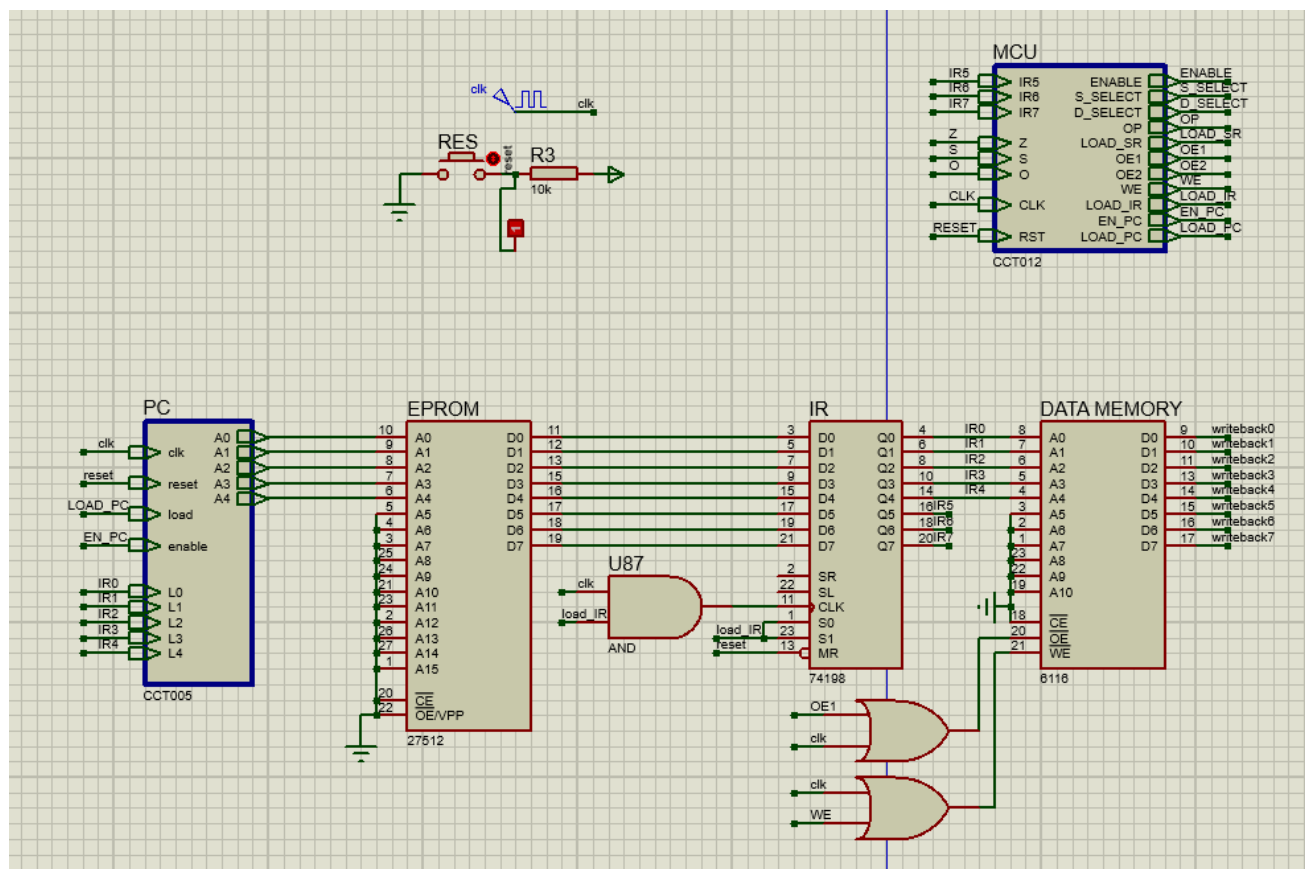
گروه ۲ - چهارشنبه عصر

عنوان آزمایش: واحد کنترل ریزبرنامه سازی شده

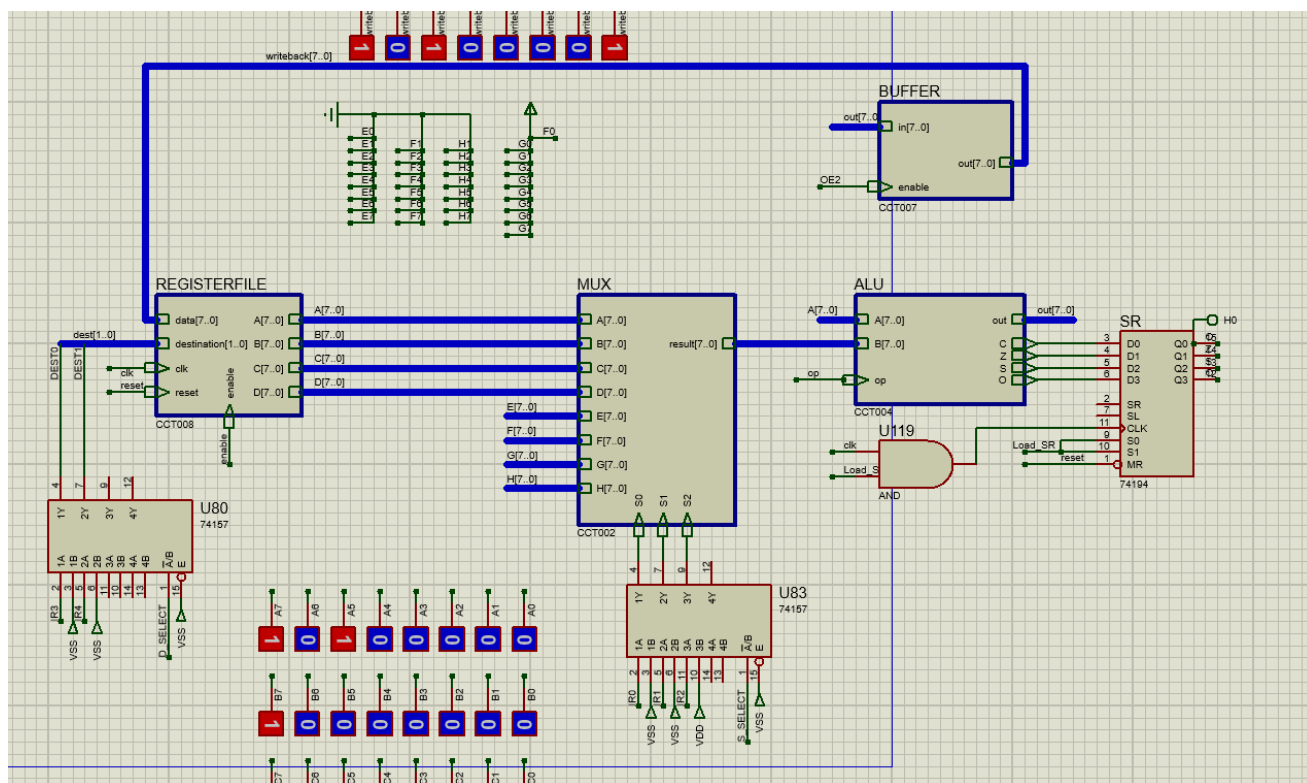
هدف آزمایش: در این آزمایش، مدار کنترل کامپیوتر ساخته شده در آزمایش های ششم، هفتم و هشتم را به صورت ریزبرنامه پذیر طراحی و پیاده سازی می کنیم.

شرح آزمایش:

در این آزمایش، کلیت مسیر داده مشابه آزمایش قبل است؛ یعنی دستور از EPROM خوانده می شود، داخل IR قرار می گیرد، سپس واحد کنترل بر اساس opcode دستور، سیگنال های لازم را برای PC، حافظه داده، ثبات ها، MUX، ALU و SR تولید می کند. بنابراین بخش هایی مثل PC، EPROM، IR، حافظه داده، Register File، MUX، Status Register و ALU تقریباً همان نقش قبلی را دارند. (شکل ۱ و ۲)



شکل ۱

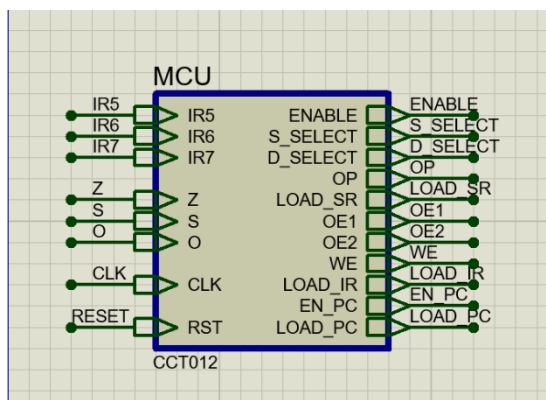


شکل ۲

تفاوت اصلی این آزمایش در نحوه تولید سیگنال‌های کنترلی است. در آزمایش قبل کنترل مدار بیشتر به صورت مستقیم و سخت‌افزاری انجام می‌شد، اما در این آزمایش واحد کنترل به شکل ریزبرنامه‌ای پیاده‌سازی شده است. به همین دلیل بلوک MCU تغییر کرده و سیگنال‌هایی مانند WE ، $OE1/OE2$ ، $LOAD_PC$ ، EN_PC ، $LOAD_IR$ و $LOAD_SR$ و انتخاب‌های مربوط به مسیر داده را تولید می‌کند.

در بخش Register File نیز ورودی‌های $dest$ و $source$ دیگر مستقیماً فقط از بیت‌های دستور گرفته نمی‌شوند، بلکه از طریق MUX انتخاب می‌شوند. دلیل این کار این است که در بعضی مراحل اجرای دستور، مقصد یا منبع باید از خود instruction بیاید، اما در بعضی مراحل داخلی ریزبرنامه، واحد کنترل نیاز دارد مقدار ثابتی را به عنوان $source/destination$ اعمال کند. بنابراین MUX باعث می‌شود انتخاب ثبات‌ها انعطاف‌پذیرتر شود و MCU بتواند در هر ریزمرحله، مسیر مناسب را کنترل کند.

در نتیجه، روند کلی اجرای برنامه مثل قبل است، اما کنترل اجرای هر دستور دیگر یک مرحله‌ای و ثابت نیست؛ بلکه به چند ریزدستور تقسیم شده و MCU در هر کلاک سیگنال‌های لازم همان ریزمرحله را فعال می‌کند. (شکل ۳)



شکل ۳

در داخل بلوک MCU، دو حافظه ROM وجود دارد که فایل‌های `msb.hex` و `lsb.hex` روی آن‌ها بارگذاری شده‌اند (شکل ۴). از آن جایی که تعداد سیگنال‌های کنترلی زیاد است، هر ریزدستور در ۱۶ بیت ذخیره شده است. بنابراین ۸ بیت پایین هر ریزدستور در فایل `lsb.hex` و ۸ بیت بالای آن در فایل `msb.hex` قرار گرفته است. به عبارت دیگر، برای هر آدرس از حافظه کنترل، خروجی دو ROM کنار هم قرار می‌گیرند و یک ریزدستور کامل ۱۶ بیتی را تشکیل می‌دهند. برای مثال، در آدرس 1 حافظه کنترل، مقدار فایل‌ها به این صورت است:

`lsb = E8`
`msb = 00`

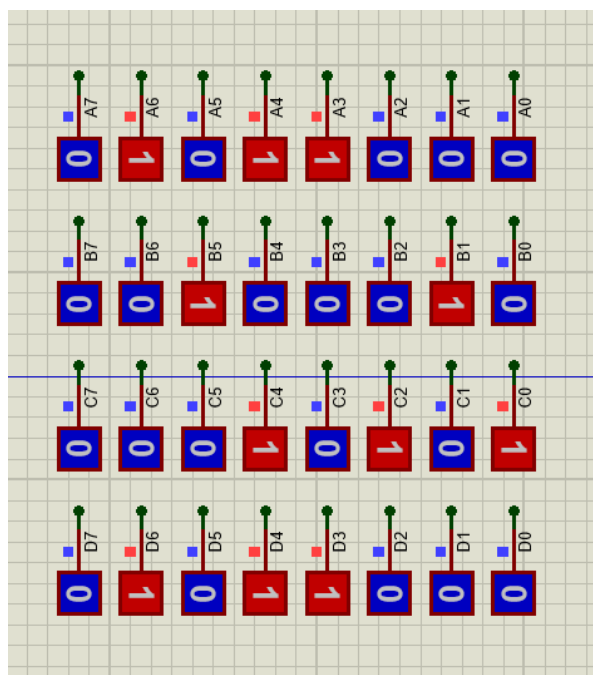
پس ریزدستور کامل برابر است با:

00E8

00E8 دستور ماشین نیست؛ یک کلمه کنترلی داخل MCU است. بطور مثال وقتی MCU این ریزدستور را می‌خواند، در این گام خروجی‌های حافظه/بافر با سیگنال‌های `OE1` و `OE2` روی باس فعال می‌شوند، اما سیگنال‌هایی مثل `LOAD_IR`، `LOAD_PC`، `EN_PC`، `WE` و عملیات ALU فعال نیستند؛ یعنی این مرحله بیشتر برای قرار دادن داده روی باس و آماده‌سازی مرحله بعدی اجرای دستور استفاده می‌شود.

هر کدام از بیت‌های این ریزدستورها به یکی از سیگنال‌های کنترل مدار مربوط هستند. برای نمونه، خروجی‌های MCU شامل سیگنال‌هایی مانند `LOAD_PC`، `EN_PC`، `LOAD_IR` و ... هستند. با فعال یا غیرفعال شدن این سیگنال‌ها، مسیر اجرای دستور در مدار مشخص می‌شود.

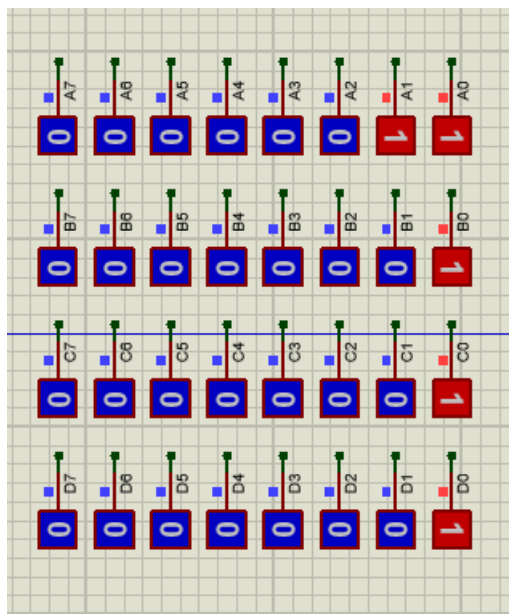
تست جمع ۱۰ جمله اول فیبوناچی (شکل ۵)



شکل ۵

جمع نهایی در رجیستر A نوشته شده و معادل با ۸۸ است که پاسخ صحیحی است.

جمع دو عدد ۳۲ بیتی: $01010101h + 00000002h = 01010103h$ (شکل ۶)



شکل ۶

پایان